

Generic SortedDictionary Collection Class in C# with Examples

In this article, I am going to discuss the **Generic SortedDictionary<TKey, TValue> Collection Class in C#** with Examples. Please read our previous article where we discussed [Generic SortedSet<T> Collection Class in C#](#) with Examples. The Generic SortedDictionary<TKey, TValue> Collection Class belongs to System.Collections.Generic namespace. At the end of this article, you will understand the following pointers with examples.

1. [What is SortedDictionary<TKey, TValue> Collection Class in C#?](#)
2. [How to Create a SortedDictionary<TKey, TValue> collection in C#?](#)
3. [How to Add Elements into a Generic SortedDictionary Collection in C#?](#)
4. [How to Access a Generic SortedDictionary<TKey, TValue> Collection in C#?](#)
5. [How to Remove Elements from a Generic SortedDictionary Collection in C#?](#)
6. [How to check the Availability of key/value Pairs in a Generic SortedDictionary Collection in C#?](#)
7. [How to Assign Values to a SortedDictionary<TKey, TValue> with Indexer in C#?](#)
8. [How to Update a SortedDictionary<TKey, TValue> Collection in C# using Indexer?](#)
9. [Using Enumerator to Iterates through SortedDictionary<TKey, TValue> Collection in C#](#)
10. [Generic SortedDictionary<TKey, TValue> Collection with Complex Type in C#](#)
11. [What is the use of the TryGetValue\(\) method of SortedDictionary Class in C#?](#)
12. [How to get all the keys and Values of a SortedDictionary in C#?](#)
13. [Differences Between SortedDictionary<TKey, TValue> and SortedList<TKey, TValue> in C#?](#)

What is SortedDictionary<TKey, TValue> Collection Class in C#?

The SortedDictionary<TKey, TValue> is a Generic Collection class in C# which is used to store the key/value pairs in the sorted form and the sorting is done based on the key. The SortedDictionary<TKey, TValue> class uses the concept of the hashtable. As it is a Generic Collection, so, it belongs to System.Collections.Generic namespace.

The keys of the SortedDictionary collection must be unique and maintains ascending order based on the key. With the help of a key, we can easily search or remove elements from the generic SortedDictionary<TKey, TValue> collection.

It is dynamic in nature which means the size of the sorted dictionary is increased automatically as we added key/value pairs to the collection. The keys of SortedDictionary<TKey, TValue> collection cannot be null but the value can be null for a reference type.

How to Create a SortedDictionary<TKey, TValue> collection in C#?

The Generic SortedDictionary<TKey, TValue> Collection class in C# provided the following Constructors that we can use to create an instance of SortedDictionary class.

1. **SortedDictionary():** It initializes a new instance of the Generic SortedDictionary class that is empty and uses the default System.Collections.Generic.IComparer implementation for the key type.
2. **SortedDictionary(IComparer<TKey> comparer):** It initializes a new instance of the Generic SortedDictionary class that is empty and uses the specified System.Collections.Generic.IComparer implementation to compare keys. The parameter comparer specifies the System.Collections.Generic.IComparer implementation to use when comparing keys, or null to use the default System.Collections.Generic.Comparer for the type of the key.

3. **SortedDictionary(IDictionary<TKey, TValue> dictionary):** It initializes a new instance of the Generic SortedDictionary class that contains elements copied from the specified System.Collections.Generic.IDictionary and uses the default System.Collections.Generic.IComparer implementation for the key type. The parameter comparer specifies the System.Collections.Generic.IDictionary whose elements are copied to the new Generic SortedDictionary. If the dictionary is null, then it will throw ArgumentNullException. If the dictionary contains one or more duplicate keys, then it will throw ArgumentException.
4. **SortedDictionary(IDictionary<TKey, TValue> dictionary, IComparer<TKey> comparer):** It initializes a new instance of the Generic SortedDictionary class that contains elements copied from the specified System.Collections.Generic.IDictionary and uses the specified System.Collections.Generic.IComparer implementation to compare keys. The parameter comparer specifies the System.Collections.Generic.IDictionary whose elements are copied to the new Generic SortedDictionary. The parameter comparer specifies the System.Collections.Generic.IComparer implementation to use when comparing keys, or null to use the default System.Collections.Generic.Comparer for the type of the key. If the dictionary is null, then it will throw ArgumentNullException. If the dictionary contains one or more duplicate keys, then it will throw ArgumentException.

Let's see how to create a SortedDictionary<TKey, TValue> collection using the SortedDictionary() constructor in C#:

Step1:

As the SortedDictionary<TKey, TValue> Collection Class belongs to System.Collections.Generic namespace, so first, we need to import the System.Collections.Generic namespace into our program as follows:

using System.Collections.Generic;

Step2:

Next, we need to create an instance of the SortedDictionary<TKey, TValue> collection class using the SortedDictionary() constructor as follows:

SortedDictionary<TKey, TValue> sortedDictionaryName = new SortedDictionary<TKey, TValue>();

How to Add Elements into a Generic SortedDictionary Collection in C#?

If you want to add a key/value pair to a Generic SortedDictionary Collection, then you need to use the Add() method provided by the Generic SortedDictionary class.

- **Add(TKey key, TValue value):** The Add(TKey key, TValue value) method is used to add an element with the specified key and value into the Generic SortedDictionary collection. Here, the parameter key specifies the key of the element to add and the parameter value specifies the element to add. The value can be null for reference types. If the key is null, then it will throw ArgumentNullException and if an element with the same key already exists in the Generic SortedDictionary, then it will throw ArgumentException.

For example, here we are creating a generic SortedDictionary collection by specifying the key as an integer and the value as a string as follows.

```
SortedDictionary<int, string> genericSortedDictionary = new SortedDictionary<int, string>();  
genericSortedDictionary.Add(1, "One");  
genericSortedDictionary.Add(3, "Three");  
genericSortedDictionary.Add(2, "Two");
```

The following statement will give us a compile-time error as we pass the key as null.

genericSortedDictionary.Add(null, "Four");

The following statement will throw a Runtime Exception as we pass the key as 2 which already exists in the collection.

```
genericSortedDictionary.Add(2, "AnyValue");
```

As the SortedDictionary class has the Add method which takes TKey and TValue as Parameters, so, we can also store key/values pair in the Generic SortedDictionary Collection using Collection Initializer as follows.

```
SortedDictionary<int, string> genericSortedDictionary = new SortedDictionary<int, string>  
{  
    { 1, "One"},  
    { 3, "Three"},  
    { 2, "Two"}  
};
```

Note: You can add the elements into the collection in random order. But they are going to be stored in the collection in Ascending order based on key.

How to Access a Generic SortedDictionary<TKey, TValue> Collection in C#?

We can access the key/value pairs of the Generic SortedDictionary<TKey, TValue> Collection in C# using three different ways. They are as follows:

Using For-Each loop to access Generic SortedDictionary Collection in C#:

We can use a for-each loop to access the key/value pairs of the Generic SortedDictionary Collection in C# as follows.

```
foreach (KeyValuePair<int, string> item in genericSortedDictionary)  
{  
    Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");  
}
```

Using For Loop to access Generic SortedDictionary Collection in C#:

We can also use a for-each loop to access the key/value pairs of the Generic SortedDictionary Collection in C# as follows. Here, we are using the ElementAt method and hence we need to include the System.Linq namespace.

```
for (int i = 0; i < genericSortedDictionary.Count; i++)  
{  
    KeyValuePair<int, string> element = genericSortedDictionary.ElementAt(i);  
    Console.WriteLine($"Key: { element.Key}, Value: { element.Value}");  
}
```

Using Index to Access Individual SortedDictionary Collection Elements in C#:

You can access the individual value of the Generic SortedDictionary Collection in C# by using the indexer. In this case, you just need to pass the key as a parameter to find the corresponding value. For example,

```
Console.WriteLine($"Key: 1, Value: {genericSortedDictionary[1]}");  
Console.WriteLine($"Key: 2, Value: {genericSortedDictionary[2]}");  
Console.WriteLine($"Key: 3, Value: {genericSortedDictionary[5]}");
```

Example to Understand How to Create a Generic SortedDictionary Collection and Add Elements in C#:

For a better understanding of how to create a Generic SortedDictionary Collection and how to add elements to the collection, and how to access the elements from the collection, please have a look at the below example.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericSortedDictionaryCollection
{
    public class Program
    {
        public static void Main()
        {
            //Creating Generic SortedDictionary Collection with key as integer
            //as string
            SortedDictionary<int, string> genericSortedDictionary = new
            SortedDictionary<int, string>();
            //Adding Elements to SortedDictionary Collection using Add Method
            //Order
            genericSortedDictionary.Add(1, "One");
            genericSortedDictionary.Add(3, "Three");
            genericSortedDictionary.Add(2, "Two");
            genericSortedDictionary.Add(5, "Five");
            genericSortedDictionary.Add(4, "Four");
            //Duplicate Key not allowed, Runtime Exception
            //System.ArgumentException: 'An entry with the same key already exists'
            //genericSortedDictionary.Add(2, "Any Value");
            //Key Cannot be null: Compile Time Error
            //genericSortedDictionary.Add(null, "Any Value");
            //Accessing Generic SortedDictionary Collection using For Each loop
            Console.WriteLine("Accessing SortedDictionary Collection using For
            loop");
            foreach (var item in genericSortedDictionary)
            {
                Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
            }
            //Accessing Generic SortedDictionary Collection using For loop
            Console.WriteLine("\nAccessing SortedDictionary Collection using For loop");
            for (int i = 0; i < genericSortedDictionary.Count; i++)
            {
                KeyValuePair<int, string> element = genericSortedDictionary.ElementAt(i);
                Console.WriteLine($"Key: { element.Key}, Value: { element.Value}");
            }

            //Accessing SortedDictionary Collection Individual Elements using
            //Indexer
        }
    }
}
```

```

        Console.WriteLine("\nAccessing SortedList Individual Elements using
Indexer");
        Console.WriteLine($"Key: 1, Value: {genericSortedDictionary[1]}");
        Console.WriteLine($"Key: 2, Value: {genericSortedDictionary[2]}");
        Console.WriteLine($"Key: 5, Value: {genericSortedDictionary[5]}");
        Console.ReadKey();
    }
}

```

Output:

```

Accessing SortedDictionary Collection using For Each loop
Key: 1, Value: One
Key: 2, Value: Two
Key: 3, Value: Three
Key: 4, Value: Four
Key: 5, Value: Five

Accessing SortedDictionary Collection using For loop
Key: 1, Value: One
Key: 2, Value: Two
Key: 3, Value: Three
Key: 4, Value: Four
Key: 5, Value: Five

Accessing SortedList Individual Elements using Key as Indexer
Key: 1, Value: One
Key: 2, Value: Two
Key: 5, Value: Five

```

Note: We are getting the output based on the ascending order of the keys when accessing the elements using For and For Each loop.

Adding Elements to Generic SortedDictionary Collection using Collection Initializer in C#:

As the Generic SortedDictionary class has the Add method which takes TKey and TValue as parameters, so we can use the collection initializer to initialize the SortedDictionary collection while creating. In the below example, we are using Collection Initializer syntax instead of the Add method to add key-value pairs into the Generic SortedDictionary Collection in C#.

```

using System;
using System.Collections.Generic;
namespace GenericSortedDictionaryCollection
{
    public class Program
    {

```

```

    public static void Main()
    {
        //Creating Generic SortedDictionary Collection with key as integer
as string
        //Adding Element using Collection Initializer
        SortedDictionary<int, string> genericSortedDictionary = new
SortedDictionary<int, string>
        {
            { 1, "One" },
            { 3, "Three" },
            { 5, "Five" },
            { 4, "Four" }
        };
        //Adding Element using Add Method
        genericSortedDictionary.Add(2, "Two");
        //Accessing Generic SortedDictionary Collection using For Each loop
        Console.WriteLine("Accessing SortedDictionary using For Each loop'
foreach (var item in genericSortedDictionary)
        {
            Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
        }
        Console.ReadKey();
    }
}

```

Output:

```

Accessing SortedDictionary using For Each loop
Key: 1, Value: One
Key: 2, Value: Two
Key: 3, Value: Three
Key: 4, Value: Four
Key: 5, Value: Five

```

How to Remove Elements from a Generic SortedDictionary Collection in C#?

The Generic SortedDictionary Collection Class in C# provides the following methods to remove elements from SortedList.

1. **Remove(TKey key):** This method is used to remove the element with the specified key from the Generic SortedDictionary. The parameter key specifies the element to remove. It returns true if the element is successfully removed; otherwise, false. This method also returns false if the key was not found in the original Generic SortedDictionary. If the key is null, then it will throw ArgumentNullException.
2. **Clear():** This method is used to remove all elements from a Generic SortedDictionary Collection.

Let us see an example to understand the above three methods of Generic SortedDictionary Collection Class in C#. Please have a look at the below example.

```

using System;
using System.Collections.Generic;
namespace GenericSortedDictionaryCollection
{
    public class Program
    {
        public static void Main()
        {
            //Creating a Generic SortedDictionary Collection with both the key
            values are of string type
            SortedDictionary<string, string> genericSortedDictionary = new
            SortedDictionary<string, string>();
            //Adding Elements to Generic SortedDictionary Collection using Add
            genericSortedDictionary.Add("Ind", "India");
            genericSortedDictionary.Add("USA", "United State of America");
            genericSortedDictionary.Add("SA", "South Africa");
            genericSortedDictionary.Add("SL", "Srilanka");
            genericSortedDictionary.Add("ENG", "England");
            //Accessing SortedDictionary Collection using For Each loop
            Console.WriteLine($"SortedDictionary Elements Count=
            {genericSortedDictionary.Count}");
            foreach (var item in genericSortedDictionary)
            {
                Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
            }
            // Remove value having key SL Using Remove() method
            genericSortedDictionary.Remove("SL");
            Console.WriteLine($"SortedDictionary Elements After Remove Metho
            {genericSortedDictionary.Count}");
            foreach (KeyValuePair<string, string> item in genericSortedDictior
            {
                Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
            }
            // Remove all key/value pairs Using Clear method
            genericSortedDictionary.Clear();
            Console.WriteLine($"After Clear Method Count=
            {genericSortedDictionary.Count}");
            Console.ReadKey();
        }
    }
}

```

Output:

```

SortedDictionary Elements Count=5
Key: ENG, Value: England
Key: Ind, Value: India
Key: SA, Value: South Africa
Key: SL, Value: Srilanka
Key: USA, Value: United State of America

SortedDictionary Elements After Remove Method Count=4
Key: ENG, Value: England
Key: Ind, Value: India
Key: SA, Value: South Africa
Key: USA, Value: United State of America

After Clear Method Count=0

```

How to check the Availability of key/value Pairs in a Generic SortedDictionary Collection in C#?

If you want to check whether the key/value pair exists or not in the Generic SortedDictionary Collection in C#, then you can use the following methods provided by the Generic SortedDictionary<TKey, TValue> class as per your requirement i.e. whether you want to search for an element by key or an element by value.

1. **ContainsKey(TKey key):** This method is used to determine whether the Generic SortedDictionary collection contains a specific key. The parameter key to locating in the Generic SortedDictionary object. It returns true if the Generic SortedDictionary collection contains an element with the specified key; otherwise, false. If the key is null, then it will throw ArgumentNullException.
2. **ContainsValue(TValue value):** This method is used to determine whether a Generic SortedDictionary contains a specific value. The parameter value to locate in the Generic SortedDictionary collection. The value can be null for reference types. It returns true if the Generic SortedDictionary Collection contains an element with the specified value; otherwise, false.

Let us understand this with an example. The following example shows how to use the both ContainsKey and ContainsValue method of the Generic SortedDictionary collection class in C#.

```

using System;
using System.Collections.Generic;
namespace GenericSortedDictionaryCollection
{
    public class Program
    {
        public static void Main()
        {
            //Creating a Generic SortedDictionary Collection with both the key
            values are of string type
            SortedDictionary<string, string> genericSortedDictionary = new
            SortedDictionary<string, string>();
            //Adding Elements to Generic SortedDictionary Collection using Add
            genericSortedDictionary.Add("Ind", "India");
            genericSortedDictionary.Add("USA", "United State of America");

```

```

genericSortedDictionary.Add("SA", "South Africa");
genericSortedDictionary.Add("SL", "Srilanka");
genericSortedDictionary.Add("ENG", "England");
//Accessing SortedDictionary Collection using For Each loop
Console.WriteLine($"SortedDictionary Elements:");
foreach (var item in genericSortedDictionary)
{
    Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
}
//Checking the key using the ContainsKey method
Console.WriteLine("\nIs Ind Key Exists : " +
genericSortedDictionary.ContainsKey("Ind"));
Console.WriteLine("Is NZ Key Exists : " +
genericSortedDictionary.ContainsKey("NZ"));

//Checking the value using the ContainsValue method
Console.WriteLine("\nIs India value Exists : " +
genericSortedDictionary.ContainsValue("India"));
Console.WriteLine("Is Bangladesh value Exists : " +
genericSortedDictionary.ContainsValue("Bangladesh"));
Console.ReadKey();
    }
}
}

```

Output:

```

SortedDictionary Elements:
Key: ENG, Value: England
Key: Ind, Value: India
Key: SA, Value: South Africa
Key: SL, Value: Srilanka
Key: USA, Value: United State of America

Is Ind Key Exists : True
Is NZ Key Exists : False

Is India value Exists : True
Is Bangladesh value Exists : False

```

How to Assign Values to a SortedDictionary<TKey, TValue> with Indexer in C#?

In order to add value to a SortedDictionary<TKey, TValue> with an indexer, we need to use square brackets after the SortedDictionary name. This is because a SortedDictionary works with key/value pairs, and we have to specify both key and value while adding the elements. The key is specified between square brackets. The syntax is given below.

```
genericSortedDictionary[key] = value;
```

For a better understanding, please have a look at the following example. In the below example, first, we have created the dictionary with a few key-value pairs. Then we added new key-value pair to the genericSortedDictionary with the indexer. Here, SA, ENG, and SL are the keys, and South Africa, England, and Srilanka are the values that correspond to each key respectively.

```
using System;
using System.Collections.Generic;
namespace GenericSortedDictionaryCollection
{
    public class Program
    {
        public static void Main()
        {
            SortedDictionary<string, string> genericSortedDictionary = new
SortedDictionary<string, string>
            {
                { "Ind", "India" },
                { "USA", "United State of America" }
            };
            genericSortedDictionary["SA"] = "South Africa";
            genericSortedDictionary["SL"] = "Srilanka";
            genericSortedDictionary["ENG"] = "England";
            //Accessing SortedDictionary Collection using For Each loop
            Console.WriteLine($"SortedDictionary Elements:");
            foreach (var item in genericSortedDictionary)
            {
                Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
SortedDictionary Elements:
Key: ENG, Value: England
Key: Ind, Value: India
Key: SA, Value: South Africa
Key: SL, Value: Srilanka
Key: USA, Value: United State of America
```

How to Update a SortedDictionary<TKey, TValue> Collection in C# using Indexer?

We already discussed that we can retrieve the value from the SortedDictionary by using the key in the indexer. In the same way, we can also use the key indexer to update an existing key-value pair in the SortedDictionary collection in C#. For a better understanding, please have a look at the below example.

```
using System;
```

```

using System.Collections.Generic;
namespace GenericSortedDictionaryCollection
{
    public class Program
    {
        public static void Main()
        {
            SortedDictionary<string, string> genericSortedDictionary = new
SortedDictionary<string, string>
            {
                { "Ind", "India" },
                { "USA", "United State of America" }
            };

            Console.WriteLine("Before Updating the Key Ind and USA");
            Console.WriteLine($"Ind: {genericSortedDictionary["Ind"]}");
            Console.WriteLine($"USA: {genericSortedDictionary["USA"]}");
            //Updating the key UK and USA using Indexer
            genericSortedDictionary["Ind"] = "India Updated";
            genericSortedDictionary["USA"] = "USA Updated";
            Console.WriteLine("\nAfter Updating the Key Ind and USA");
            Console.WriteLine($"Ind: {genericSortedDictionary["Ind"]}");
            Console.WriteLine($"USA: {genericSortedDictionary["USA"]}");
            Console.ReadKey();
        }
    }
}

```

Output:

```

Before Updating the Key Ind and USA
Ind: India
USA: United State of America

After Updating the Key Ind and USA
Ind: India Updated
USA: USA Updated

```

Using Enumerator to Iterates through SortedDictionary<TKey, TValue> Collection in C#:

The SortedDictionary<TKey, TValue>.GetEnumerator Method is used to get an enumerator that iterates through the SortedDictionary<TKey, TValue>.

Syntax: **public** **System.Collections.Generic.SortedDictionary<TKey, TValue>.Enumerator**
GetEnumerator();

Return Value: This method returns a SortedDictionary<TKey, TValue>.Enumerator for the SortedDictionary<TKey, TValue>.

For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
using System.Collections.Generic;
namespace GenericSortedDictionaryCollection
{
    public class Program
    {
        public static void Main()
        {
            SortedDictionary<string, string> genericSortedDictionary = new
SortedDictionary<string, string>
            {
                { "Ind", "India" },
                { "USA", "United State of America" },
                { "UK", "United Kingdom" }
            };
            // To get an IDictionaryEnumerator for the SortedDictionary
            IDictionaryEnumerator myEnumerator = genericSortedDictionary.GetEn
            // If MoveNext passes the end of the collection, the enumerator is
            // after the last element in the collection and MoveNext returns f
            while (myEnumerator.MoveNext())
            {
                Console.WriteLine($"Key: {myEnumerator.Key} and Value:
{myEnumerator.Value}");
            }

            Console.ReadKey();
        }
    }
}
```

Output:

```
Key: Ind and Value: India
Key: UK and Value: United Kingdom
Key: USA and Value: United State of America
```

Generic SortedDictionary<TKey, TValue> Collection with Complex Type in C#:

As of now, we have used the built-in data types such as int, string, etc with Generic SortedDictionary Collection Class. Now, let us see how to create a Generic SortedDictionary collection using Complex types. Let us create a class called Student and then let us create a SortedDictionary collection of Student types and also add duplicate elements.

```

using System;
using System.Collections.Generic;
namespace GenericSortedDictionaryDemo
{
    class Program
    {
        static void Main()
        {
            //Here we are creating a genericSortedDictionary whose key is int
            is Student
            SortedDictionary<int, Student> genericSortedDictionary = new
            SortedDictionary<int, Student>
            {
                { 101, new Student(){ ID = 101, Name = "Anurag", Branch="CSE"}
                { 104, new Student(){ ID = 104, Name = "Pranaya", Branch="ETC"}
                { 103, new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}
                { 102, new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"}
            };
            //The following Statement will give you Runtime Exception as the k
            already exists
            //System.ArgumentException: 'An entry with the same key already ex
            //genericSortedDictionary.Add(101, new Student() { ID = 101, Name
            Branch = "CSE" });
            //Accessing Generic SortedDictionary Collection using For Each loc
            Console.WriteLine("Generic SortedDictionary Collection Elements");
            foreach (KeyValuePair<int, Student> item in genericSortedDictionary)
            {
                Console.WriteLine($"Key: { item.Key}: ID: { item.Value.ID}, Na
            item.Value.Name}, Branch: { item.Value.Branch}");
            }
            Console.ReadKey();
        }
    }
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Branch { get; set; }
    }
}

```

Output:

```

Generic SortedDictionary Collection Elements
Key: 101: ID: 101, Name: Anurag, Branch: CSE
Key: 102: ID: 102, Name: Mohanty, Branch: CSE
Key: 103: ID: 103, Name: Sambit, Branch: ETC
Key: 104: ID: 104, Name: Pranaya, Branch: ETC

```

What is the use of the TryGetValue() method of SortedDictionary Class in C#?

This is one of the important methods of SortedDictionary collection class in C#. This method takes two parameters, one is the key and the other one is the value. The value is of out type parameter. If the key exists in the SortedDictionary, then it will return true and the value of that associated key is stored on the output variable.

If you are not sure if a key is present or not in the SortedDictionary, then you can use the TryGetValue() method to get the value from a SortedDictionary because if you are not using TryGetValue then in that case you will get KeyNotFoundException.

For a better understanding, please have a look at the below example. In the first TryGetValue method, we are passing the key as 102 and out variable i.e. std102. As we can see key 102 is present in the SortedDictionary, so, this method will return true and the associated value will be populated in the std102 variable. And as the method returns true the body of the if condition gets executed and you can see the student data in the console window.

In the second TryGetValue method, we are passing the key as 115 and out variable i.e. std115. As we can see the key 115 is not present in the SortedDictionary, so, this method will return false, and hence the value will not be populated in the std115 variable, and as the method returns false the else part of the if condition gets executed, and that you can see in the console window.

```
using System;
using System.Collections.Generic;
namespace GenericSortedDictionaryCollection
{
    class Program
    {
        static void Main()
        {
            //Here we are creating a genericSortedDictionary whose key is int
            is Student
            SortedDictionary<int, Student> genericSortedDictionary = new
            SortedDictionary<int, Student>
            {
                { 101, new Student(){ ID = 101, Name = "Anurag", Branch="CSE"}
                { 104, new Student(){ ID = 104, Name = "Pranaya", Branch="ETC"}
                { 103, new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}
                { 102, new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"}
            };
            //Accessing Generic genericSortedDictionary Collection using For &
            Console.WriteLine("Generic genericSortedDictionary Elements");
            foreach (KeyValuePair<int, Student> item in genericSortedDictionary)
            {
                Console.WriteLine($"Key: { item.Key}: ID: { item.Value.ID}, Name: {
item.Value.Name}, Branch: { item.Value.Branch}");
            }
            Student? std102;
            if (genericSortedDictionary.TryGetValue(102, out std102))
            {
```

```

        Console.WriteLine("\nStudent with Key = 102 is found in the
SortedDictionary");
        Console.WriteLine($"ID: {std102.ID}, Name: {std102.Name}, Brar
{std102.Branch}");
    }
    else
    {
        Console.WriteLine("\nStudent with Key = 102 is not found in tl
SortedDictionary");
    }
    Student? std115;
    if (genericSortedDictionary.TryGetValue(105, out std115))
    {
        Console.WriteLine("\nStudent with Key = 102 is found in the
SortedDictionary");
        Console.WriteLine($"ID: {std115.ID}, Name: {std115.Name}, Brar
{std115.Branch}");
    }
    else
    {
        Console.WriteLine("\nStudent with Key = 105 is not found in tl
SortedDictionary");
    }
    Console.ReadKey();
}
}
public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}
}

```

Output:

```

Generic genericSortedDictionary Elements
Key: 101: ID: 101, Name: Anurag, Branch: CSE
Key: 102: ID: 102, Name: Mohanty, Branch: CSE
Key: 103: ID: 103, Name: Sambit, Branch: ETC
Key: 104: ID: 104, Name: Pranaya, Branch: ETC

Student with Key = 102 is found in the SortedDictionary
ID: 102, Name: Mohanty, Branch: CSE

Student with Key = 105 is not found in the SortedDictionary

```

Note: If you are not sure if a key is present or not in the SortedDictionary, then you can use the TryGetValue() method to get the value from a SortedDictionary because if you are not using TryGetValue

then in that case you will get `KeyNotFoundException`.

How to Get all the Keys and Values of a SortedDictionary in C#?

To get all the keys in the `SortedDictionary` we have to use the `Keys` property of the `SortedDictionary` object. To get all the values of a `SortedDictionary`, first, we need to get the keys, then we need to get the values using the keys. Even if you only want the values, then you can use the `Values` property of the `SortedDictionary` object. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections.Generic;
namespace GenericSortedDictionaryCollection
{
    class Program
    {
        static void Main()
        {
            //Here we are creating a genericSortedDictionary whose key is int
            SortedDictionary<int, Student> genericSortedDictionary = new
            SortedDictionary<int, Student>
            {
                { 101, new Student(){ ID = 101, Name = "Anurag", Branch="CSE"}},
                { 104, new Student(){ ID = 104, Name = "Pranaya", Branch="ETC"}},
                { 103, new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}},
                { 102, new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"}},
            };
            Console.WriteLine("All Keys in Student SortedDictionary");
            foreach (int key in genericSortedDictionary.Keys)
            {
                Console.WriteLine(key + " ");
            }
            // Once you get the keys, then get the values using the keys
            Console.WriteLine("\nAll Keys and values in Student SortedDictionary");
            foreach (int key in genericSortedDictionary.Keys)
            {
                var student = genericSortedDictionary[key];
                Console.WriteLine($"Key: {key}, Values = ID: {student.ID}, Name: {student.Name}, Branch: {student.Branch}");
            }
            //To get all the values in the dictionary use Values property
            Console.WriteLine("\nAll Student objects in Student SortedDictionary");
            foreach (Student student in genericSortedDictionary.Values)
            {
                Console.WriteLine($"Values = ID: {student.ID}, Name: {student.Name}, Branch: {student.Branch}");
            }
            Console.ReadKey();
        }
    }
}
```

```

public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}

```

Output:

```

All Keys in Student SortedDictionary
101
102
103
104

All Keys and values in Student SortedDictionary
Key: 101, Values = ID: 101, Name: Anurag, Branch: CSE
Key: 102, Values = ID: 102, Name: Mohanty, Branch: CSE
Key: 103, Values = ID: 103, Name: Sambit, Branch: ETC
Key: 104, Values = ID: 104, Name: Pranaya, Branch: ETC

All Student objects in Student SortedDictionary
Values = ID: 101, Name: Anurag, Branch: CSE
Values = ID: 102, Name: Mohanty, Branch: CSE
Values = ID: 103, Name: Sambit, Branch: ETC
Values = ID: 104, Name: Pranaya, Branch: ETC

```

Differences Between SortedDictionary<TKey, TValue> and SortedList<TKey, TValue> in C#?

The SortedDictionary<TKey, TValue> Generic Class is a binary search tree with $O(\log n)$ retrieval, where n is the number of elements in the dictionary. It is very much similar to the [SortedList<TKey, TValue> Generic Class](#). The two classes have similar object models, and both have $O(\log n)$ retrieval. Where the two classes differ is in memory use and speed of insertion and removal as follows:

1. SortedList<TKey, TValue> uses less memory than SortedDictionary<TKey, TValue>.
2. SortedDictionary<TKey, TValue> has faster insertion and removal operations for unsorted data: $O(\log n)$ as opposed to $O(n)$ for SortedList<TKey, TValue>.
3. If the collection is populated all at once from sorted data, SortedList<TKey, TValue> is faster than SortedDictionary<TKey, TValue>.

Each key/value pair can be retrieved as a KeyValuePair<TKey, TValue> structure, or as a DictionaryEntry through the nongeneric IDictionary interface.

C# SortedDictionary<TKey, TValue> Collection Class Summary:

1. In SortedDictionary, the key must be unique (duplicate keys are not allowed) and the key cannot be null.
2. In SortedDictionary, the value can be null when the type of the value is of reference type.
3. The operations of insertion of elements and removing the elements can be faster using SortedDictionary class.
4. The same type of key and value pairs can be stored using SortedDictionary class.
5. The total number of key and value pairs that the SortedDictionary class can hold is the capacity of the SortedDictionary class.
6. The sorting is done in ascending order by SortedDictionary class.
7. The SortedDictionary in C# is dynamic in nature which means the size of the SortedDictionary increases as per the need.

In the next article, I am going to discuss the [Generic LinkedList<T> Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **Generic SortedDictionary<TKey, TValue> Collection Class in C#** with Examples. I hope this Generic SortedDictionary<TKey, TValue> Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.